

Universidad Carlos III de Madrid
Department of Computer Science

Cybersecurity Engineering Labs

Student's Guide

v1.0
September 2026

Authors, Licensing, and Copyright

Copyright © 2026 Juan Tapiador

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/4.0/>.

Third-party material

This collection of lab exercises incorporates materials developed by third parties for educational purposes, including:

- Palo Alto Networks Unit 42 Wireshark Workshop (video training and PCAP datasets).
- The Metasploitable 2 vulnerable virtual machine provided by Rapid7.

Contributors

These lab materials were designed and compiled by the course coordinator, Juan Tapiador, for the Cybersecurity Engineering course taught in the Computer Science program at UC3M. Over the years, other course instructors have also made valuable comments, suggestions, and contributed with exercises:

- José Miguel Colmena
- Roberto Fuentes
- Antonio Nappa
- José Miguel Moreno

Table of Contents

Authors, Licensing, and Copyright	1
Third-party material	1
Contributors	1
Table of Contents	2
Lab 1. Authentication in Linux	4
Basic user management in Linux	4
Preparatory readings	4
Exercises	4
Cracking passwords with John the Ripper	4
Preparatory readings	5
Prerequisites	5
JtR cheatsheet	5
Exercises	7
A Large-scale cracking experiment	9
Further experimenting with password strength (optional)	10
Pluggable Authentication Modules (PAM)	10
Preparatory readings	10
Prerequisites	11
Exercises	11
Lab 2. Access Control in Linux	13
Linux permissions	13
Preparatory readings	13
Exercises	13
ACLs and capabilities	14
Preparatory readings	14
Exercises	14
Lab 3. Analyzing Network Traces with Wireshark	17
Preparatory readings	17
Wireshark cheatsheet	17
Wireshark introductory lab	18
Exercises: UI customization	19
Exercises: Packet filtering	19
Exercises: Host identification	19
Exercises: Traffic inspection (I): Windows telemetry	20
Exercises: Traffic inspection (II): FTP	20
Exercises: Traffic inspection (III): SMTP and POP	20
Exercises: Traffic inspection (IV): A downloaded file	20
Security analysis of network traces	20

Exercises: Analyzing trace1	20
Exercises: Analyzing trace2	21
Exercises: Analyzing trace3	21
Exercises: Analyzing trace4	21
Lab 4. Network Discovery and Port Scanning	23
Preparatory readings	23
Exercises	23
Basic exploitation	24
Lab 5. Web Security	25
Exercises: Command injection	25
Exercises: SQL injection	25
Exercises: File upload	26
Exercises: XSS	26
Exercises: CSRF	26
Lab 6. Malware Analysis	28
Prepare the analysis environment	28
Isolating the VM	28
Getting the samples	28
Defanging	28
Changing the extension	29
Password-protect	29
Defanging other IOCs	29
Exercises	29
Appendix. Exercise Tracker	31

Lab 1. Authentication in Linux

Lab goals:

1. Acquire basic notions of user management in Linux.
2. Learn how user passwords are created, stored, and used.
3. Have a hands-on first contact with password strength using cracking tools.
4. Study alternative authentication methods (PAM).

You need:

- A Linux distribution where you have root privileges
- Make sure that you have installed the **mkpasswd** command and john the ripper
 - Use your favorite package manager, e.g., **sudo apt-get install john**

Basic user management in Linux

The goal of this first part of the lab is to introduce some hands-on experience with the basic tools to manipulate user passwords.

Preparatory readings

Read this material before coming to the lab:

- Patrick Louis. [A Compendium of Access Control on Unix-Like OSes](#). This is a comprehensive document that we will use throughout the course. For this lab, read these two sections:
 - [Proving Who We Are](#)
 - [The Password And Group Files](#)

Exercises

Ex.1. Add a user (useradd). Check out the different flags and options for the useradd command.

Ex.2. Inspect the `/etc/passwd` file. Describe all fields in each entry. Note that passwords are not stored there, but in the `/etc/shadow` file. Study this file, too.

Cracking passwords with John the Ripper

The goal of this first part of the lab is to introduce the use of John the Ripper and crack some simple passwords.

Preparatory readings

Read this material before coming to the lab. These are also useful references for the rest of the lab:

- Cracking modes: <https://www.openwall.com/john/doc/MODES.shtml>
- Examples: <https://www.openwall.com/john/doc/EXAMPLES.shtml>
- Rules: <https://www.openwall.com/john/doc/RULES.shtml>
- Configuration file: <https://www.openwall.com/john/doc/CONFIG.shtml>
- Wordlists: <https://github.com/kkrypt0nn/wordlists>

Prerequisites

For this exercise you will need large dictionaries of words (wordlists) to guide password cracking efforts. Security-oriented Linux distributions such as Kali or Parrot OS typically have them already installed. For other Linux distributions, you need to install the wordlist package and unzip the rockyou wordlist:

```
$ sudo apt update && sudo apt install wordlists
$ cd /usr/share/wordlists/
$ sudo gunzip rockyou.txt.gz
```

JtR cheatsheet

John might need a little help to detect the format of your hashes. If you are greeted with a “No password hashes loaded” message, use `--format` to specify the hash format. For example:

```
$ john --format=crypt --mask='?d?d?d?d' hashes.txt
```

Masks

?l	Lowercase letters	abcdefghijklmnopqrstuvwxyz
?u	Uppercase letters	ABCDEFGHIJKLMNOPQRSTUVWXYZ
?d	Digits	0123456789
?s	Special characters	<>!"£\$%^&*()_+{}[]#~\ /?
?a	All of the above	(ASCII printable characters)
?A	Full ASCII	(All ASCII characters)
?h	Hex digit	0123456789abcdef

?H	Uppercase hex digit	0123456789ABCDEF
?w	Hybrid	(Current item from a wordlist)
?1	Placeholder	(From ?1 to ?9, defined with flag --1='...')

Rules

```
# Reverse order ("hello" -> "olleh")
```

```
[List.Rules:Reverse]
```

```
r
```

```
# Prepend ("hi" -> "@hi")
```

```
[List.Rules:Prepend]
```

```
A0"@"
```

```
# Leave the item unaltered ("hi" -> "hi")
```

```
# Duplicate ("hi" -> "hihi")
```

```
[List.Rules:Duplicate]
```

```
:
```

```
d
```

```
# Remove first character ("hello" -> "ello")
```

```
# Duplicate last character ("hello" -> "hell")
```

```
# Replace 1st and 3rd characters ("hello" -> "*e*lo")
```

```
[List.Rules:Censor]
```

```
\[
```

```
\]
```

```
o0* o2*
```

```
# Concatenate its reverse ("hello" -> "helloolleh")
```

```
[List.Rules:Reflect]
```

```
f
```

```
# All to lowercase ("HeLlO" -> "hello")
```

```
[List.Rules:Lower]
```

```
l
```

```
# Capitalize ("hELLo" -> "Hello")
```

```
[List.Rules:Capitalize]
c

# Uppercase and append 123 ("foo" -> "F00123")
[List.Rules:Append]
u Az"123"

# Toggle only 1st, 2nd or 3rd characters
# Toggle 2 out of the 3 first characters
# Toggle all first 3 characters
[List.Rules:Toggle]
T0
T1
T2
T0T1
T0T2
T1T2
T0T1T2
```

Exercises

Ex.3. Use the unshadow command to extract usernames and passwords from both files.

Ex.4. Explore formats supported by JtR.

Ex.5. One very useful way of creating hashes (instead of changing the password of the user you just created) is through the mkpasswd command. Create some passwords and inspect them.

Ex.6. You can list all brute-force (incremental) strategies by running:

```
$ ls /usr/share/john/*.chr
```

Launch an attack that tries all ASCII combinations up to 5 characters long.

Note: You can check the status of JtR at any point using the show flag:

```
$ john --show hashes.txt
```

Ex.7. Launch an attack that generates all possible numerical PINs of 4 digits

Ex.8. Try wordlists, e.g., rockyou.

Ex.9. Repeat the previous exercise but using a custom dictionary of common words. check how fast JtR is at cracking them. See then what happens when you do lower/upper case variations, digits, etc.

Ex.10. Try more complex passwords and use masks to crack them. For example, do foo123 and try a mask that makes it super easy to crack it:

Ex.11. You can also use a custom alphabet, which could be a subset of the standard alphabet or one that includes diacritics and other symbols. Find out how to try 3 characters long passwords composed of printable ASCII with diacritics.

Ex.12. Investigate how JtR can make combinations of words using very simple masks. Hint: try the `stdout` flag combined with the `min-length` and/or `max-length` options.

Ex.13. Rules are transformation directives applied to a wordlist to automatically generate common password variations during a dictionary attack. Instead of storing massive wordlists, rules mutate simple words into likely passwords on the fly. Investigate and experiment with rules to try the following common password mutations:

- Case Toggling: Changes password to Password or PASSWORD.
- Appends & Prepends: Adds digits or symbols, like turning admin into admin123!.
- Character Substitution (Leet Speak): Replaces letters with numbers/symbols, like p@ssw0rd.
- Structural Edits: Reverses, duplicates, or truncates words (e.g., passpass or ssaw).

Rules can be passed as command-line parameters or are defined using a custom syntax inside the `/etc/john/john.conf` configuration file under sections like `[List.Rules:Wordlist]`. While not mandatory, it's better to not modify it and add new rules in `/usr/share/john/john-local.conf` instead. Note the location of this file changing depending on the GNU/Linux distribution.

Try adding them to the configuration file for this exercise.

Ex.14. [OPTIONAL].The `john.conf` file serves as the main configuration hub for John the Ripper. It is partitioned into specific headers (e.g., `[Section.Type:Name]`) that dictate global defaults, define various rule sets, and manage the technical behavior of cracking algorithms. Its main sections are:

- `[Options]` and `[System]`: These sections manage global parameters, including paths for default wordlists, CPU thread allocation, process priority, and automatic save intervals for checkpoints.

- `[List.Rules:*)`: Defines built-in and custom mutation sequences (such as `[List.Rules:Wordlist]` or `[List.Rules:Jumbo]`) that specify how characters are transformed during dictionary-based attacks.
- `[Incremental:*)`: Used to configure brute-force modes (like `[Incremental:ASCII]`), allowing you to set length limits, character sets, and link to `.chr` statistical files.
- `[List.External:*)`: Provides a way to use C-style functions compiled at runtime for sophisticated custom logic, such as specialized candidate generation or hash filtering.
- `[Markov:*)`: Configures statistical Markov-chain attacks by setting probability thresholds and weighting character sequences to favor realistic password patterns.
- `[Disabled:*)`: A mechanism to turn off particular hash formats or attack methods to improve performance and streamline the cracking process.

Study the `john.conf` file in your system and try some advanced configurations.

A Large-scale cracking experiment

Ex.15. `ForoMotos.com`, a well-known website among Spanish-speaking internet users, has suffered a security breach, and the data of its more than 180,000 users is being sold in various cybercrime-related marketplaces. You have obtained a partial version of the leak that only includes pairs of usernames and password hashes.

`Meneate.net`, a link-sharing social network quite popular among `ForoMotos` users, has also been the victim of another data leak. In this case, you have managed to get a dataset with username and hash pairs for its more than 220,000 users.

The goal of this exercise is simple: crack as many passwords as possible from both websites using John the Ripper.

Hints:

- In the early years of `ForoMotos.com`, the website defaulted to generating 5-character (later increased to 6) lowercase alphanumeric passwords. Later on, they were auto-generated including uppercase letters and requiring at least one special symbol.
- The minimum length for `ForoMotos.com` passwords is 4 characters.
- Both websites are populated by Spanish-speaking users, which will surely be reflected in the passwords they use.
- `ForoMotos.com` has a subforum dedicated to hacking where script kiddies use the jargon specific to their elite hacking skills.
- Some users of the website often reuse the same password on multiple sites.

Further experimenting with password strength (optional)

Ex.16. The objective of this exercise is to deepen the study of password strength and the use of password-cracking tools. To that end, each lab group must generate several sets of passwords with different complexity levels and empirically study the effort required to crack them using different strategies.

1. Generation of password datasets of different complexity. Generate 25 datasets of 100 passwords each. Each dataset must contain passwords of similar complexity according to a certain combination of complexity metrics. Include at least the following datasets:
 - 5 datasets of passwords of length 3...7 formed only by lowercase letters.
 - 5 datasets of passwords of length 3...7 formed only by uppercase letters.
 - 5 datasets of passwords of length 3...7 formed only by digits.
 - 5 datasets of passwords of length 3...7 formed by alphanumeric characters (both upper- and lowercase) and symbols.
 - For the remaining 5 datasets, use strategies that involve words taken from a dictionary.
2. Repeat the previous steps (i.e., generate another 25 datasets) using a hashing algorithm that is very different in computational cost from the one used for the first 25. For example, use md5crypt for the first group and SHA-512 for the second.
3. Design at least 5 different strategies to crack passwords using John the Ripper. Each strategy is defined by a cracking mode and, where applicable, the corresponding parameters, rules, or configuration file.
4. Run each of the strategies designed in step 3 against each of the datasets generated in step 1. Give John the Ripper a limited amount of time and stop it after that time, because for some datasets it may require a very long time on your machine. Once the process has finished for each dataset, obtain the following metrics:
 - Percentage of passwords cracked.
 - Mean and median of the time required to crack a password.
5. Organize the results in tables. Each table should contain one row per dataset and one column for each of the metrics described in step 4. Inspect them visually and analytically, and describe the main conclusions you draw from this study.

Plugable Authentication Modules (PAM)

Preparatory readings

Read this material before coming to the lab:

- Patrick Louis. [A Compendium of Access Control on Unix-Like OSes](#). This is a comprehensive document that we will use throughout the course. For this lab, read this section:
 - [Pluggable Authentication Module \(PAM\)](#)
- [PAM\(8\) man page](#).

Prerequisites

For this set of exercises you will need

- gcc
- libpam0g-dev, and
- pamtester

Exercises

Ex.17. Create a dummy PAM application config at `/etc/pam.d/pamtest` to analyze how control flags are evaluated in order. First use the following stack:

```
auth    sufficient    pam_permit.so
auth    required      pam_deny.so
```

Run `pamtester pamtest <username> authenticate` and observe the output. Now, invert the flags:

```
auth    required      pam_deny.so
auth    sufficient    pam_permit.so
```

- (a) Is it possible for authentication to succeed in this second configuration? Why or why not?
- (b) If you replace `sufficient` with `requisite` in a failing module at the top of a stack, will subsequent modules be executed?

Ex.18. Your task in this exercise is to restrict `su` access to specific groups and configure brute-force lockout using `pam_wheel.so` and `pam_faillock.so`. To do so:

- Modify `/etc/pam.d/su` to allow root access only to members of the `wheel` group.
- Edit `/etc/pam.d/common-auth` to lock accounts for 15 minutes after 3 consecutive failed attempts using `pam_faillock.so`.
-

Answer these questions:

- (a) Is it possible for a non-`wheel` user to authenticate via `su` if `pam_wheel.so` is configured with the optional control flag `while` `pam_unix.so` is set to `required`?

(b) If `pam_faillock.so` is placed after `pam_unix.so` as sufficient, will failed attempts still increment the lockout counter?

Ex.19. You now have to chain a system script into the authentication flow to alert on root access. To achieve this, write a bash script (for example, `/usr/local/bin/pam_alert.sh`) that appends the user, terminal, and timestamp to `/var/log/pam_alerts.log`. Then add

```
auth optional pam_exec.so /usr/local/bin/pam_alert.sh
```

to `/etc/pam.d/sshd`.

If `/usr/local/bin/pam_alert.sh` exits with code 1 (failure) and `pam_exec.so` is set to required, is it possible for a user with a valid password to successfully log in via SSH?

Ex.20. Create a custom PAM module, named `pam_passcode.c`, that prompts the user for a hardcoded PIN before standard login processing. Is it possible for your custom module to inject environment variables into the user's session upon successful authentication? Which PAM API function achieves this?

Lab 2. Access Control in Linux

Lab goals:

1. Learn standard Linux permissions and their administrative tools.
2. Implement POSIX ACLs for fine-grained access control.
3. Analyze Linux capabilities and process privilege management.

You need:

- A Linux distribution where you have root privileges
- Make sure that you have installed the **acl** package
 - Use your favorite package manager, e.g., **sudo apt-get install acl**

Linux permissions

The goal of this first part of the second lab is to introduce some hands-on experience with the basic tools to manipulate user accounts and their privileges.

Preparatory readings

Read this material before coming to the lab.

- Patrick Louis. [A Compendium of Access Control on Unix-Like OSes](#). Read this section:
 - [Basic File Permission](#)
- RedHat. [How to manage users and groups in Linux](#).
- RedHat. [How to manage Linux permissions for users, groups, and others](#).
- RedHat. [Linux permissions: SUID, SGID, and sticky bit](#).
- Linux Handbook. [chattr Command Examples](#).

Exercises

The following exercises will create several groups and user accounts on your system. If you are using a virtual machine, you can take a screenshot before doing this lab and then restore it to the original state. Alternatively, you can manually remove all accounts, groups, files and directories after finishing the lab.

Ex.21. Create five user accounts that we will use later to experiment with permissions and ownership by using these accounts. (You can set them all with the same password for easy access.)

Ex.22. Create a group for each of the following departments: sales, marketing, human resources, IT, engineering. For each group, create a directory in the /home directory.

Configure ownership for the directories such that each directory belongs to root and to the corresponding group. Finally adjust permissions so that users in a group have access to the corresponding group directory but not to other departments' directories. Run various commands to test that it works as intended.

Ex.23. Assign each of the previously created five users to each of the groups. Use the touch command to create files in each directory with different users to test if the permissions are correctly configured.

Ex.24. Create a file named immutable.txt in /home/marketing. Make it immutable. Try to delete it (even with the root account!) and check that it cannot be removed. Use lsattr to list the attributes of the file.

Ex.25. Create a file named /home/marketing/addonly.txt. Put a few lines of text in it. Make it append-only. Test that other users (depending on the permissions) can add but not remove it. Discuss how these examples override the write permission.

Ex.26. Create a file and change the A attribute. Touch it several times and check that the access time remains unchanged.

ACLs and capabilities

Preparatory readings

Read this material before coming to the lab.

- Patrick Louis. [A Compendium of Access Control on Unix-Like OSes](#). Read these three sections:
 - [POSIX.1e/2c Access Control Lists](#)
 - [POSIX.1e/2c Capabilities](#)
 - [POSIX Capabilities on Linux](#)
- RedHat. [An introduction to Linux Access Control Lists \(ACLs\)](#).

Exercises

Prepare your Linux machine so that it has the required users, groups and directories needed for this lab. You can use the following commands:

```
# Create groups
sudo groupadd --gid=1001 prof-ai
sudo groupadd --gid=1002 prof-cyber
sudo groupadd --gid=1003 prof-web
sudo groupadd --gid=2001 student-ai-cyber
sudo groupadd --gid=2002 student-ai-web
```

```
sudo groupadd --gid=2003 student-cyber-web
sudo groupadd --gid=501 ai
sudo groupadd --gid=502 cyber
sudo groupadd --gid=503 web

# Create users
sudo useradd --uid=1001 --gid=1001 --create-home --shell=/bin/bash prof-ai
sudo useradd --uid=1002 --gid=1002 --create-home --shell=/bin/bash prof-cyber
sudo useradd --uid=1003 --gid=1003 --create-home --shell=/bin/bash prof-web
sudo useradd --uid=2001 --gid=2001 --create-home --shell=/bin/bash
student-ai-cyber
sudo useradd --uid=2002 --gid=2002 --create-home --shell=/bin/bash
student-ai-web
sudo useradd --uid=2003 --gid=2003 --create-home --shell=/bin/bash
student-cyber-web
sudo usermod -a --groups=ai,cyber student-ai-cyber
sudo usermod -a --groups=ai,web student-ai-web
sudo usermod -a --groups=cyber,web student-cyber-web
sudo chmod 700 /home/student-* /home/prof-*

# Create course directories
sudo mkdir -p /courses/{cyber,ai,web}
sudo chown prof-ai:ai /courses/ai
sudo chown prof-cyber:cyber /courses/cyber
sudo chown prof-web:web /courses/web
sudo chmod 750 /courses/{ai,cyber,web}
sudo chmod g+s /courses/{ai,cyber,web}
```

Make sure you understand what the previous commands do before doing the next exercises.

Ex.27. Switch to user `prof-cyber`. Try to create a file in `/courses/web`. Can you do it?

Ex.28. Try to create a file in `/courses/cyber`. What are the default permissions, owning user and owning group? There's something weird about the group, what is it and why did it happen?

Ex.29. Course directories are intended to be read-only for students and only modifiable by their respective instructors. However, the instructor for the AI course has asked us (the sysadmin) to create a discussion board file called `forum.txt` that students from the AI course can modify. Create the file `/courses/ai/forum.txt` so that enrolled students can also modify it.

Ex.30. Switch to user `student-ai-cyber` and verify that you can write to the file. Can you delete it? How can you prevent students from deleting the file but still write to it?

- Ex.31. List the permissions of the `/courses/ai` directory. Why is the T uppercase?
- Ex.32. It turns out that the `forum.txt` file we created was by default granted `o+r` permissions. How can we make sure that newly created files have by default no permissions in the other class? (Hint: use an ACL.)
- Ex.33. Switch to user `prof-web`. The user `student-cyber-web` is yet to submit their final assignment of the course. We want to publish the solution to this assignment for the rest of the class except for this user. How can you create a file in `/courses/web/solution.txt` that the previous user cannot read while still being readable for the rest of the web group?
- Ex.34. The university has asked us to create a file in each course called `announcements.txt` that can only be modified by the `adm` group. Not even the professors should have write access. How can we achieve this? IMPORTANT: As some of our instructors are paranoid about getting their material stolen, we need to make sure that users in the `adm` group cannot read the rest of the course contents.
- Ex.35. The cybersecurity students need to use `tcpdump` for an assignment. However, the executable does not work without root privileges. Why? Why doesn't this happen with the `ping` command? How can we let users run this command without granting them root privileges?

Lab 3. Analyzing Network Traces with Wireshark

Lab goals:

1. Introduce wireshark as a powerful tool to perform network analysis.
2. Review the operation of basic network protocols at all layers, including ARP, DHCP, IP, TCP, UDP, DNS, SMB, HTTP, FTP, SMTP, POP.

You need:

- The latest stable release of Wireshark installed. Get it from <https://www.wireshark.org>

Preparatory readings

Palo Alto's threat intelligence Unit 42 has a fantastic tutorial that introduces Wireshark in the context of security investigations. It is a 5-part video and the slides and pcap files are also available:

- Tutorial: <https://unit42.paloaltonetworks.com/wireshark-workshop-videos/>
- Slides and pcaps: <https://github.com/pan-unit42/Wireshark-workshop>

We will be demonstrating parts of this workshop during the first part of this lab to gain some hands-on experience with Wireshark.

In addition, this tutorial about creating some basic filters is very useful:

- <https://unit42.paloaltonetworks.com/using-wireshark-display-filter-expressions/>

Wireshark cheatsheet

Basic usage

The most basic filtering is by protocol. You can just type the protocol name, such as:

ip, dhcp, arp, udp, tcp, dns, http, tls, ...

Logical operators

- AND: and or &&
- OR: or or ||
- NOT: not or !

Comparison operators

- Equal: eq or ==
- Not equal: ne or !=
- Greater than: gt or >

- Greater than or equal to: ge or >=
- Less than: lt or <
- Less than or equal to: le or <=

Filters

Each protocol has several fields that can be used for filtering. Examples include (see the manual for a complete reference):

- ip.addr == 192.168.0.1
- ip.src == 192.168.0.1 or ip.dst == 192.168.0.2
- ip.dst == 192.128.4.0/24
- (ip.src == 192.168.0.1 and ip.dst != 10.0.0.1) or ip.src == 10.0.0.1
- tcp.port == 443
- tcp.srcport == 25
- tcp.stream == 123
- frame.len > 450
- eth.addr == aa:bb:cc:dd:ee:ff
- http.host == "example.com"

Tracking individual conversations

Right-click a packet and then select **Follow > TCP Stream** (or **UDP Stream**). It opens a separate window showing the conversation chronologically. You should see the appropriate stream filter in the text bar.

Exporting objects

To automatically reassemble and export transferred objects to a file, click **File > Export Objects > HTTP** (or **SMB, ...**)

Statistics

The **Statistics** menu provides useful statistical summaries of the traffic capture, including a breakdown of packet counts by protocol hierarchy, volume traffic graphs, web requests (including browsed domains and endpoints), IP destinations and TCP/UDP ports, etc.

Decryption

With the proper keys, Wireshark can decrypt encrypted traffic under standard protocols such as TLS or Wi-Fi. We will not study this capability during this lab but you are encouraged to explore it as a complementary activity.

Wireshark introductory lab

This first part of the lab uses some exercises from Unit 42's workshop to introduce Wireshark, demonstrating how to perform basic UI customization, packet filtering, host identification, and traffic filtering.

Exercises: UI customization

For this exercise, use the file `workshop-part-02-01.pcap` (password: `unit42`).

Ex.36. Remove the following columns from the default Wireshark configuration: protocol, frame number and frame length. Then add two columns for source and destination ports. Modify time to show just UTC date without microseconds.

Ex.37. Show full MAC addresses (e.g., `HewlettP_1c:47:ae` becomes `00:08:02:1c:47:ae`).

Ex.38. Add column showing hostname extracted from HTTP, TLS or DNS.

Ex.39. View Packet Diagram.

Exercises: Packet filtering

For this exercise, use the file `workshop-part-02-01.pcap` (password: `unit42`).

Ex.40. List all DNS responses.

Ex.41. Write and save a filter for listing HTTP requests without SSDP traffic.

Ex.42. Write a filter for listing TCP SYN segments

Exercises: Host identification

For this exercise, use the files `workshop-part-02-01.pcap`, `workshop-part-02-02.pcap`, and `workshop-part-02-03.pcap` (password: `unit42`).

Ex.43. Find a DHCP request and inspect it: (a) source and destination MAC/IP addresses (b) requested IP address, and (c) source hostname.

Ex.44. Find the reply to previous DHCP request: (a) source and destination MAC/IP addresses, (b) leased IP address and lease time, (c) domain name.

Ex.45. Find ARP traffic for the leased IP address.

Ex.46. Find SMB traffic for the leased IP address.

Ex.47. Find HTTP traffic from the leased IP address and identify (a) the operating system, and (b) the web browser version.

Exercises: Traffic inspection (I): Windows telemetry

For this exercise, use the file `workshop-part-04-04.pcap` (password: `unit42`).

Ex.48. Find connected USB device metadata in capture.

Ex.49. Find the weather forecast for today: date, time, and temperature.

Exercises: Traffic inspection (II): FTP

For this exercise, use the file `workshop-part-04-12.pcap` (password: `unit42`).

Ex.50. Inspect the FTP commands data channel and report: (a) the server port number, (b) the username and password used for authentication, and (c) the requested file(s).

Ex.51. Extract the downloaded file(s) from the capture.

Exercises: Traffic inspection (III): SMTP and POP

For this exercise, use the file `workshop-part-04-15.pcap` (password: `unit42`).

Ex.52. Find SMTP traffic and identify the remote server hostname. Is the traffic in cleartext or encrypted?

Ex.53. Find POP traffic and identify the remote server hostname. Is the traffic in cleartext or encrypted? Report: (a) the username and password used for authentication, (b) the number of available messages, (c) the number of retrieved messages, and (d) the sender and receiver addresses from retrieved message(s).

Exercises: Traffic inspection (IV): A downloaded file

For this exercise, use the file `workshop-part-04-16.pcap` (password: `unit42`).

Ex.54. Extract the XLSX file downloaded from the shared folder and identify: (a) the full filename, and (b) the client and server MAC and IP addresses.

Security analysis of network traces

The goal of this second part of the lab is to learn how to identify behaviors in a network trace using a traffic analysis tool.

Exercises: Analyzing trace1

Load the trace file `trace_1.pcap` into Wireshark and answer the following questions.

- Ex.55. What is the most prevalent application protocol in the trace?
- Ex.56. What are the IP addresses and ports of the client and the server?
- Ex.57. What type and version of server is running?
- Ex.58. Write a filter that displays the first TCP packet of each flow in the trace.
- Ex.59. Looking at the overall trace, what do you think is the client's goal?

Exercises: Analyzing trace2

Load the trace file `trace_2.pcapng` into Wireshark and answer the following questions.

- Ex.60. Describe how many TCP and UDP conversations are in the trace and how many hosts are involved.
- Ex.61. Analyzing the overall trace, what is the host with IP address 192.168.5.51 doing?
- Ex.62. List all the techniques that the host with IP address 192.168.5.51 is using.
- Ex.63. What ports are open on the host with IP address 192.168.5.20?

Exercises: Analyzing trace3

Load the trace file `trace_3.pcapng` into Wireshark and answer the following questions.

- Ex.64. Write a filter that displays all HTTP conversations in the trace.
- Ex.65. Describe all the information you can find out about the HTTP server and client.
- Ex.66. Obtain the passwords (in plaintext) from the HTTP Authorization headers contained in the trace.

Exercises: Analyzing trace4

Load the trace file `trace_4.pcap` into Wireshark and answer the following questions.

- Ex.67. This network trace contains a malware infection of a Windows host. Provide the date and time of this activity.
- Ex.68. Indicate the IP and MAC addresses and the hostname of the infected host.
- Ex.69. What is the domain name and IP address of the compromised website?

Ex.70. What is the domain name and IP address of the website from which the malware is downloaded?

Ex.71. What browser does the infected Windows host use?

Ex.72. After infection, the malware uses HTTP communications. Which method (GET or POST) does it use?

Lab 4. Network Discovery and Port Scanning

Lab goals:

1. Identify host enumeration and scanning techniques using Nmap.
2. Perform hands-on network discovery and port scanning against a vulnerable machine.
3. Understand the security risks posed by network services.

You need:

1. Install the nmap package in your Linux distribution.
2. Download the [Metaexploitable2](#) distribution and create a VM with it in VirtualBox (or the hypervisor you are using). The credentials for this machine are

`msfadmin:msfadmin`

3. Create a virtual network on the 192.168.56.1/24 IP address range. Attach both machines to this network (Machine > Settings > Network → Attached-to: Host-only Adapter, and select the virtual network name from the drop down menu).
4. Run both machines and give each of them an IP address in the range above.

Preparatory readings

Read chapters 1, 3, 4, 5, 7 and 8 of the [nmap book](#). Get familiar with the core capabilities for network enumeration and port scanning available in nmap.

Exercises

Ex.73. Run a ping scan in the 192.168.56.1/24 subnetwork. List all hosts that are reported as up.

Ex.74. Scan all TCP ports of the target host (the Metaexploitable2 machine) using two different methods.

Ex.75. Now try to scan all UDP ports. Does it take longer than the TCP scanning? Why?

Ex.76. Find the device type, OS and version of the target host.

Ex.77. Run a service detection only for those ports determined as open before.

Ex.78. Run it again using all service probes and compare the result with the default service detection.

Basic exploitation

This second part of the lab illustrates the security risks posed by vulnerable, malicious or misconfigured services using very basic exploitation techniques.

Ex.79. Notice that the target machine is running a well known bind shell (a backdoor) over some TCP port. Connect to it and check that this is a backdoor that gives you root access. Use the privileges to steal the password files from the target machine. Crack it and report as many passwords as you can.

Ex.80. TCP ports 512, 513, and 514 are known as “r” services. If they are misconfigured to allow remote access from any host (a `.rhosts + +` configuration), remote attackers could log in. Try to exploit this.

Ex.81. Note that NFS is up and listening on port 2049 (you can also asking the portmapper for a list of services: `rpcinfo -p <ip_address>`). Read the `showmount` man page and find out what exports the victim machine have and how can be exploited.

Ex.82. Metasploitable2 runs vsftpd, a popular FTP server, on port 21. This particular version was [backdoored](#) (this was 2011 and supply-chain attacks were not as popular as they have become lately). Read about the backdoor and exploit it.

Lab 5. Web Security

Lab goals:

1. Practice web security concepts studied in class.
2. Perform hands-on web security attacks against a vulnerable machine, including
 - a. Cross-site Scripting (XSS)
 - b. Command injection
 - c. SQL injection
3. Understand the security risks posed by Internet-facing web services.

You need:

1. The same network configuration used in Lab 4. We assume the Metaexploitable2 machine has IP address 192.168.56.4.

Exercises: Command injection

Ex.83. From the Linux machine connected to the virtual network, open a browser and connect to `http://192.168.56.4` (change the IP address by the one where your Metaexploitable2 machine is running) Use the credentials `admin/password` to log in. Click on DVWA Security and select Low. Click on Command Execution and then on View Source. You do not need to have an in-depth knowledge of PHP to understand how the script works. Look for the functions and statements in the code and make sure you understand how it works. Describe what vulnerability this code has and exploit it using a simple example.

Ex.84. How can you exploit this vulnerability to gain a basic reverse shell on the victim machine?

Ex.85. Try to exploit this vulnerability but using the Medium security level.

Ex.86. Try to exploit this vulnerability but using the High security level.

Exercises: SQL injection

Ex.87. Select SQL injection with a Low security level. Inspect the source code and describe what vulnerability it contains. Perform a basic SQL query to retrieve all tables from the MySQL database. (Hint: first find out what this is called in MySQL.)

Ex.88. Using the output from the previous exercise, identify the table containing all users and retrieve the username and password for each user. (Optional: create a text file with all username:password pairs and crack some with `john --format=raw-MD5 pwd.txt`.)

Ex.89. Repeat the previous exploit attempt (just a simple SQL injection) using the Medium security level.

Ex.90. Try to exploit this vulnerability but using the High security level.

Exercises: File upload

Ex.91. Click on the File Upload functionality on the main left-hand navigation sidebar of the DVWA interface with Low security level. Study the code and exploit the vulnerability that it contains.

Ex.92. Exploit the vulnerability in the previous exercise with a Medium security level.

Ex.93. Try to exploit it now using the High security level.

Exercises: XSS

Ex.94. Click on the Stored XSS functionality on the main left-hand navigation sidebar of the DVWA interface with Low security level. Study the code and exploit the vulnerability that it contains.

Ex.95. Repeat the previous exercise to recover the session cookie.

Ex.96. Change security level to Medium and inspect the code. Can you still exploit the vulnerability?

Ex.97. Try to exploit it now using the High security level.

Exercises: CSRF

Ex.98. Click on the CSRF functionality on the main left-hand navigation sidebar of the DVWA interface with Low security level. Change the password and check how the password is changed (look at the HTTP method and how the username and the new password is sent). Using this information, open a terminal window and use curl to change the password of an arbitrary user. Is it possible to do it? Why not?

Ex.99. Repeat the previous exercise using the cookie you stole before using the reflected XSS attack.

Lab 6. Malware Analysis

Lab goals:

1. Analyze the code of some malware samples and understand their overall intent
2. Get a first contact with basic techniques used by malware for initial access persistence, elevation of privilege, C2, and action on the objective.

You need:

1. A virtual machine configured as described below

WARNING! These are working samples. Manipulate them with care. DO NOT run them unless you know what you are doing.

Prepare the analysis environment

You will be handling live malware in this lab. There are several steps that you must follow to avoid detonating the sample in your host.

Isolating the VM

Make sure the VM you use for opening the samples is completely isolated, i.e., it has no network connection or any shared folder with the host machine.

Watch out other mechanisms that could be used to move information from/to the host machine. One classic example is the shared clipboard.

If you need to install some tool or getting a sample, do the following:

- You can temporarily attach the network adapter to NAT if you need to upgrade the analysis machine or install some additional tool. Make sure you do that from a safe snapshot. Take a snapshot after the process and put again the network adapter to host-only.
- Use the shared folders functionality to move samples from your host computer to the virtual machine. Remove the shared folder before proceeding with the analysis.

Getting the samples

Move to your analysis machine the sample `sample0.zip`.

Defanging

This term refers to actions taken by the analyst to avoid detonating the sample by accident. They include the following.

Changing the extension

A very common strategy to avoid accidentally running an executable Windows sample (.exe) is to append some meaningless extension to the file name that does not have executable properties, such as .mal or .sample. Windows use the extension to determine how to open a file. By adding a non-executable extension, the system won't know how to run it and will instead show a dialog. This step should be applied to any executable file, including not only the usual suspects (e.g., .exe, .js, .dll, etc) but also documents that could have embedded code.

Password-protect

A standard convention in the industry is to password-protect any malware specimen, with the default password being infected. The idea of using a password is not to keep users out, but to make sure that the analyst knows this is a malware sample. This is the most common way of exchanging malware specimens.

Defanging other IOCs

The idea of “disarming” an executable so that it doesn't accidentally run is a basic safety precaution that can be applied to other malicious artefacts, such as IP addresses and URLs. The convention adopted by the industry to do so is to put separators, such as colons (:) and dots (.) in square brackets. For example:

1. The IP address 192.168.1.16 would be defanged as 192[.]168[.]1[.]16
2. The domain https://www.example.com would become https[:]www[.]example[.]com

Exercises

Unzip sample0.zip and load the file in an editor.

- Ex.100. Take a quick look at the sample. What type of file is it? Run the file command on it. What is the output, and why?
- Ex.101. Analyze lines 4-19. Describe what these commands do and what the attacker goal is. Is there anything odd about them?
- Ex.102. Lines 21-36 kill a number of processes. Investigate what processes with these names could be and why the sample might want to kill them.
- Ex.103. What does the command in line 38 do? What is the intent of doing this?
- Ex.104. Describe what the command in line 42 does. What is the attacker's goal? Can you tell anything about the password hash?

- Ex.105. Analyze lines 44-45. Identify what specific MITRE ATT&CK technique is named after this behavior.
- Ex.106. Line 47 is messing with the DNS resolution. What is the malware intent here? What MITRE ATT&CK technique would you choose to better describe this behavior?
- Ex.107. Lines 63-136 drop and execute a bash script in /tmp/\$BOT. Copy the code of this embedded script into a separate file and open it in the editor. Study the code and answer these questions:
- What protocol is the bot using for communicating with the C2?
 - What is the \$NICK variable? Analyze how it is built and how it is used later in the code. Do you think the way it is constructed is a robust choice?
 - Analyze the main C2 loop, focusing on how the messages from the C2 server are being parsed. Describe the structure of the C2 messages received by the bot that start with PRIVMSG.
 - If you identify a remote device infected with this bot, can you exploit the knowledge gained after decoding the C2 message structure and send the bot a command that will be executed?
- Ex.108. The commands in lines 146-147 install two command-line tools. What functionality do they offer?
- Ex.109. Lines 149-160 of the malware sample contain an infinite loop that uses the two tools installed above to infect other systems. Describe what it does and what vulnerability is exploited to compromise target systems.
- Ex.110. What would be good Indicators of Compromise (IoC) for this malware sample?

Appendix. Exercise Tracker

Lab	Exercises				
Lab 1	☐ Ex.1	☐ Ex.2	☐ Ex.3	☐ Ex.4	
		☐ Ex.5			
	☐ Ex.6	☐ Ex.7	☐ Ex.8	☐ Ex.9	
		☐ Ex.10			
Lab 2	☐ Ex.11	☐ Ex.12	☐ Ex.13	☐ Ex.14	☐ Ex.15
	☐ Ex.16	☐ Ex.17	☐ Ex.18	☐ Ex.19	☐ Ex.20
Lab 3	☐ Ex.21	☐ Ex.22	☐ Ex.23	☐ Ex.24	☐ Ex.25
	☐ Ex.26	☐ Ex.27	☐ Ex.28	☐ Ex.29	☐ Ex.30
	☐ Ex.31	☐ Ex.32	☐ Ex.33	☐ Ex.34	☐ Ex.35
	☐ Ex.36	☐ Ex.37	☐ Ex.38	☐ Ex.39	☐ Ex.40
	☐ Ex.41	☐ Ex.42	☐ Ex.43	☐ Ex.44	☐ Ex.45
	☐ Ex.46	☐ Ex.47	☐ Ex.48	☐ Ex.49	☐ Ex.50
	☐ Ex.51	☐ Ex.52	☐ Ex.53	☐ Ex.54	☐ Ex.55
Lab 4	☐ Ex.56	☐ Ex.57	☐ Ex.58	☐ Ex.59	☐ Ex.60
	☐ Ex.61	☐ Ex.62	☐ Ex.63	☐ Ex.64	☐ Ex.65
	☐ Ex.66	☐ Ex.67	☐ Ex.68	☐ Ex.69	☐ Ex.70
	☐ Ex.71	☐ Ex.72			
	☐ Ex.73	☐ Ex.74	☐ Ex.75	☐ Ex.76	☐ Ex.77
	☐ Ex.78	☐ Ex.79	☐ Ex.80	☐ Ex.81	☐ Ex.82
Lab 5	☐ Ex.83	☐ Ex.84	☐ Ex.85	☐ Ex.86	☐ Ex.87
	☐ Ex.88	☐ Ex.89	☐ Ex.90	☐ Ex.91	☐ Ex.92
	☐ Ex.93	☐ Ex.94	☐ Ex.95	☐ Ex.96	☐ Ex.97
	☐ Ex.98	☐ Ex.99			
Lab 6	☐ Ex.100	☐ Ex.101	☐ Ex.102	☐ Ex.103	☐ Ex.104
	☐ Ex.105	☐ Ex.106	☐ Ex.107	☐ Ex.108	☐ Ex.109
	☐ Ex.110				